LINUX ADMINISTRATION

Sushant Aher

Jr. DevOps Engineer

INDEX

Linux User Management

Linux user management is a fundamental responsibility of a System Administrator or DevOps Engineer. Proper user management ensures system security, access control, accountability, and efficient resource utilization.

Below is a structured overview of Linux user management concepts and commands.

1. Understanding Users in Linux

In Linux, every process runs under a specific user account. Users are mainly categorized into:

1. Root User

- The superuser with complete system access.
- User ID (UID) = 0
- Can modify system files, install packages, manage users, and control services.

2. System Users

- Created by the system for running services (e.g., nginx, mysql).
- Usually have no login shell.
- UID typically below 1000 (varies by distribution

3. Regular Users

- Created for human users.
- Used for daily tasks.
- UID typically starts from 1000.

2. Important User Management Files

Linux stores user information in the following files:

`/etc/passwd`

Stores basic user information:

- Username
- UID
- GID
- Home directory
- Default shell

Example entry:
sushant:x:1001:1001:Sushant:/home/sushant:/bin/bash

`/etc/shadow`

Stores encrypted passwords and password policies.

`/etc/group`

Stores group information.

3. Creating Users

Create a New User:
sudo useradd username
Create User with Home Directory:
sudo useradd -m username
Create User with Specific Shell:
sudo useradd -m -s /bin/bash username
Set Password :
sudo passwd username

4. Modifying Users

Change Username
sudo usermod -l newname oldname

Change Home Directory
sudo usermod -d /new/home -m username

Add User to Group

sudo usermod -aG groupname username
⚠ Important: Always use `-aG` to append group membership. Without `-a`, existing groups may be removed.

5. Deleting Users

Delete User (Keep Home Directory):
sudo userdel username
Delete User with Home Directory :
sudo userdel -r username

6. Group Management

Create Group:
sudo groupadd groupname
Delete Group :
sudo groupdel groupname
Add User to Group :
sudo usermod -aG groupname username
Check User Groups
groups username

7. Switching Users

Switch User :

su - username
Execute Command as Another User
sudo -u username command

Password Aging in Linux

Password aging is a security mechanism in Linux that enforces password expiration policies. It ensures that users change their passwords periodically, reducing the risk of compromised credentials being used for long periods.

Password aging policies are especially important in enterprise environments to maintain compliance and strengthen system security.

1. What is Password Aging?

Password aging defines:

Minimum number of days before a password can be changed
Maximum number of days before a password expires
Warning period before password expiration
Inactive period after password expiry

These settings are managed using the `chage` command and stored in:

/etc/shadow

2. Viewing Password Aging Information

To check password aging details for a user:
chage -l username

3. Setting Password Expiration Policies

Set Maximum Password Validity (e.g., 90 days)

sudo chage -M 90 username

Set Minimum Days Before Password Change
sudo chage -m 7 username

Set Warning Days Before Expiration
sudo chage -W 7 username

Set Account Expiration Date
sudo chage -E 2026-12-31 username

4. Default Password Aging Settings

/etc/login.defs

Important parameters:

- `PASS_MAX_DAYS` – Maximum password validity
- `PASS_MIN_DAYS` – Minimum days between changes
- `PASS_WARN_AGE` – Warning days before expiration

Example:

PASS_MAX_DAYS 90
PASS_MIN_DAYS 7
PASS_WARN_AGE 7

5. Locking and Unlocking Accounts

Lock a User Account :
sudo passwd -l username

Unlock a User Account

sudo passwd -u username

Linux Firewall Management

Firewall management in Linux is a critical part of system security. A firewall controls incoming and outgoing network traffic based on predefined security rules. It helps protect servers from unauthorized access, brute-force attacks, and network-based threats.

In Linux, firewall management is primarily handled using:

- `iptables` (legacy but powerful)
- `firewalld` (dynamic firewall manager)
- `ufw` (Uncomplicated Firewall – Ubuntu-based systems)

1. Understanding Firewall Concepts

Before managing firewalls, it is important to understand key concepts:

Inbound Traffic – Incoming network connections to your server
Outbound Traffic – Outgoing traffic from your server
Ports – Logical endpoints (e.g., 22 for SSH, 80 for HTTP)
Protocols – TCP, UDP, ICMP

2. Firewall Using UFW (Ubuntu/Debian)

`ufw` is beginner-friendly and widely used in Ubuntu systems.

sudo ufw status
sudo ufw enable
sudo ufw disable

3. Firewall Using firewalld (RHEL/CentOS/Rocky/AlmaLinux)

Check Firewall Status
sudo systemctl status firewalld

Start and Enable Firewall

sudo systemctl start firewalld

sudo systemctl enable firewalld

Check Active Zones

sudo firewall-cmd --get-active-zones

Allow a Service (Permanent)

sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload

5. Commonly Opened Ports in Production

22 → SSH
80 → HTTP
443 → HTTPS
3306 → MySQL
5432 → PostgreSQL
8080 → Application servers
Always open only required ports.

6. Firewall Best Practices

Allow only necessary ports.
Disable unused services.
Use SSH key-based authentication.
Restrict SSH access by IP when possible.
Use separate zones for public and internal networks.
Regularly audit firewall rules.
Log dropped packets for monitoring.

7. Checking Open Ports

ss -tulnp

netstat -tulnp

Linux Package Management

Package management in Linux is the process of installing, updating, configuring, and removing software packages from a system. It ensures software is managed efficiently, securely, and consistently across environments.

Every Linux distribution uses a specific package management system.

1. What is a Package?

A package is a compressed file that contains:

Application binaries
Configuration files
Dependencies
Metadata (version, architecture, description)
Package managers automatically handle dependencies, making software installation easier and more reliable.

2. Package Management in Debian/Ubuntu (APT)

Ubuntu and Debian-based systems use:

`apt`
`apt-get`

Update Package Repository

sudo apt update

Upgrade Installed Packages

sudo apt upgrade

Install a Package:
sudo apt install nginx

Remove a Package :
sudo apt remove nginx

Remove with Configuration Files:
sudo apt purge nginx

Search for a Package:

apt search nginx

Show Package Information :

apt show nginx

Changing Permissions in Linux

Permission management is a fundamental concept in Linux system administration. It controls who can read, write, or execute files and directories, ensuring system security and proper access control.

Understanding file permissions is essential for DevOps engineers, system administrators, and security professionals.

1. Understanding Linux File Permissions

Each file and directory in Linux has three types of permissions:
Read (r) → Permission to view file contents
Write (w) → Permission to modify the file
Execute (x) → Permission to run the file as a program

Permissions are assigned to three categories of users:
User (u) → File owner
Group (g) → Users in the file's group
Others (o) → All other users

2. Viewing File Permissions

ls -l
Example output:
-rwxr-xr-- 1 user group 4096 Mar 03 file.sh

Breakdown:

`-` → File type
`rwx` → Owner permissions
`r-x` → Group permissions
`r--` → Others permissions

3. Changing Permissions Using chmod

The `chmod` command is used to change file permissions.
There are two methods:
Symbolic mode
Numeric (octal) mode

. Symbolic Mode
Add Permission
chmod u+x file.sh

Numeric (Octal) Mode
Each permission has a numeric value:
Read (r) = 4
Write (w) = 2
Execute (x) = 1

Example:
chmod 755 file.sh

4. Changing Ownership

Permissions are often managed together with ownership.
Change Owner
sudo chown username file.txt

Change Group
sudo chown :groupname file.txt

Change Owner and Group Together
sudo chown username:groupname file.txt

5. Best Practices

Follow the principle of least privilege.
Avoid using 777 permissions in production.
Restrict write access on sensitive files.
Audit file permissions regularly.
Use group-based access instead of giving permissions to “others”.